

EXPERIMENT-1

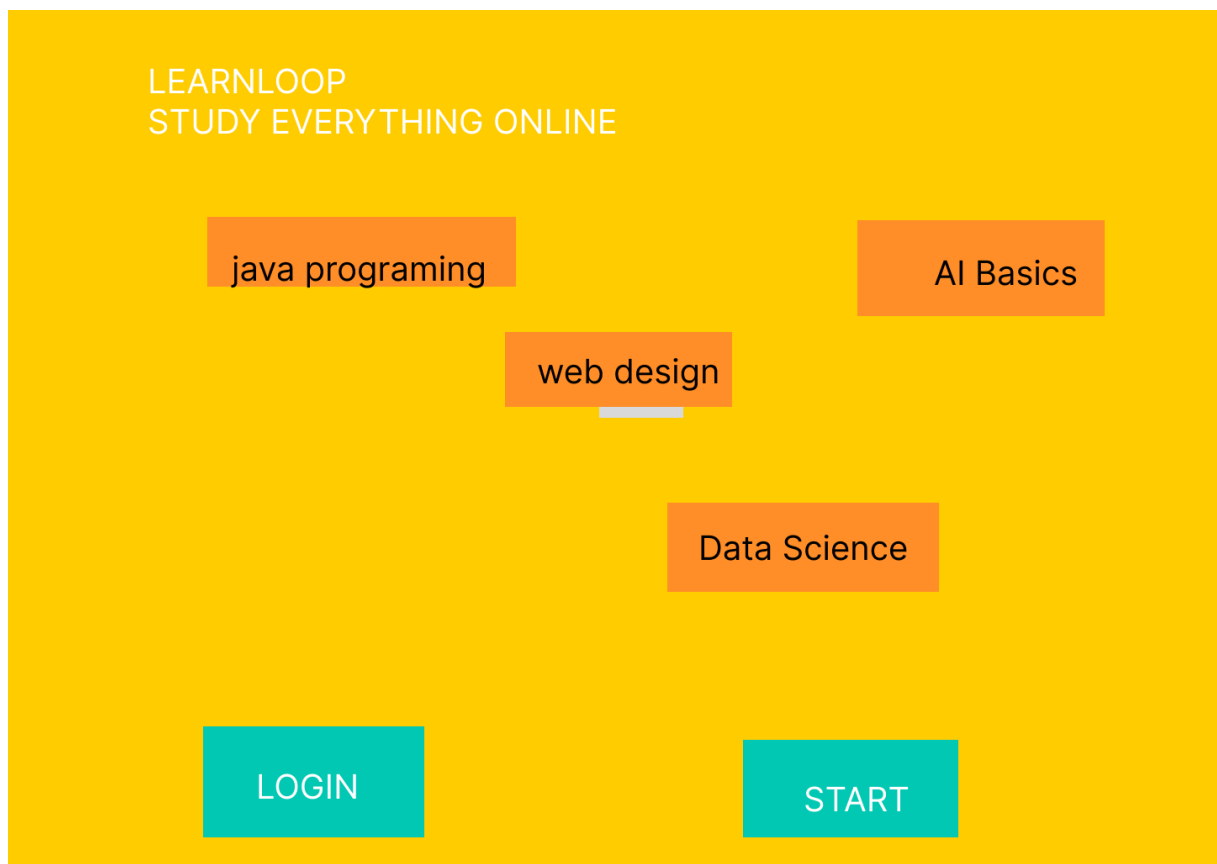
ROLL NO:240701584

NAME: Venkateshwaran . M

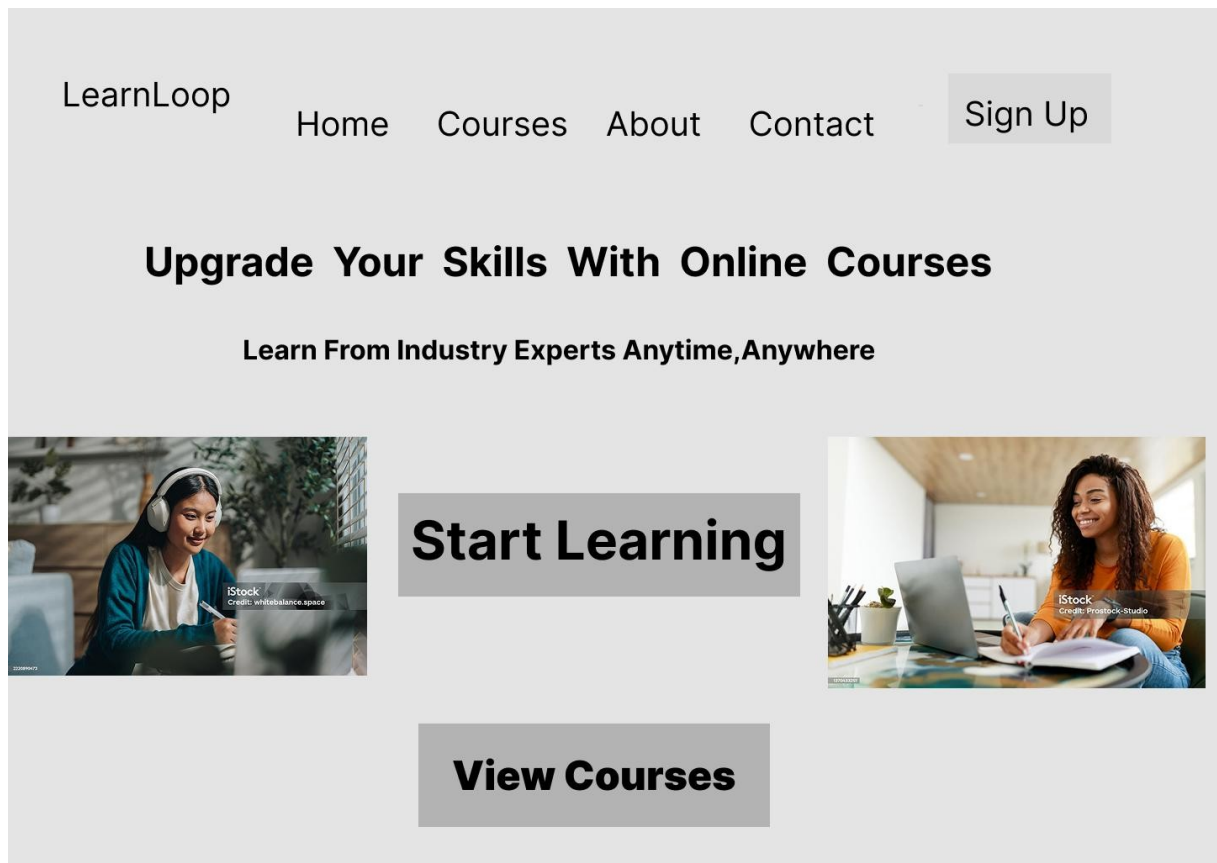
USER INTERFACE AND DESIGN

COMPARISON OF UI DESIGN BETWEEN TWO FIGMA CREATION

BAD WEBSITE:



GOOD WEBSITE:



BAD WEBSITE:

1. Design Style:

Cluttered and inconsistent layout.

Uses very bright and mixed colors that distract the user.

Elements are placed randomly without proper alignment or spacing.

2. Content:

Shows many course options (Java, Web Design, Data Science, AI Basics) without clear structure.

Text and buttons are not organized, making it confusing.

Important information is not highlighted properly.

3. Navigation:

Buttons like “Login” and “Start” are not clearly positioned.

No proper menu or clear navigation flow.

User may find it difficult to understand where to click first.

4. Purpose:

Attempts to show learning content but fails to guide the user clearly.

The design may confuse new users instead of helping them start learning.

GOOD WEBSITE:

1. Design Style:

Clean, simple, and well-structured layout.

Proper use of colors, fonts, and spacing.

Images and text are aligned neatly, giving a professional look.

2. Content:

Clearly presents the main goal: “Start Learning” and “Upgrade Your Skills.”

Includes well-placed images and readable text.

Course and action buttons like “View Courses” are easy to notice.

3. Navigation:

A clear top menu (Home, Courses, About, Contact, Sign up).

Buttons are visible and placed where users expect them.

Users can easily understand how to move through the website.

4. Purpose:

Focuses on guiding students to start learning online.

Creates a user-friendly and engaging learning platform interface.

Key Differences:

Focus:

The bad website is cluttered and confusing, while the good website focuses on clarity and user guidance.

Design Aesthetics:

The bad design uses random placement and harsh colors, whereas the good design is clean, balanced, and professional.

Content Depth:

The bad UI shows content without structure, while the good UI highlights important sections and actions.

Target Audience:

The bad UI may confuse beginners, while the good UI is suitable for students who want an easy and smooth learning experience.

EXERCISE-2

NAME: Venkateshwaran . M

ROLLNO:240701584

Design a UI where users recall visual elements (e.g., icons or text chunks). Evaluate the effect of chunking on user memory.

FRAME 1: INSTRUCTION PAGE

Analysis of the Attention and Focus Test – Instruction Screen

This screen represents the Instruction Phase of the Attention and Focus Test, where users are prepared for the task by clearly explaining what they need to observe and how they should respond. The instruction page applies principles of chunking and focused attention to reduce cognitive overload and improve task understanding.

Additionally, the presence of a START button clearly indicates the next action, making navigation intuitive. Decorative puzzle visuals subtly reinforce the idea of cognitive processing without distracting the user. Overall, this instruction screen effectively prepares users for the test by combining clarity, simplicity, and chunking principles to enhance attention and focus.

ATTENTION AND FOCUS TEST

Instructions :

- 1. You will see a screen with many mixed symbols.*
- 2. A target symbol will be given (for example: ★ Star).*
- 3. Observe the screen carefully for a few seconds.*
- 4. Focus only on the target symbols.*
- 5. Ignore all other items.*
- 6. After viewing, select the symbols you remember seeing.*
- 7. Accuracy is more important than speed*



START

FRAME 2: CHUNKING PHASE

Analysis of the Memory Recall Task – Chunking Phase Screen

This screen represents the Chunking Phase of the Memory Recall

Task, where users are briefly shown a structured set of visual items that they must observe and remember. The main objective of this phase is to enhance short-term memory by presenting information in organized chunks, rather than as random elements.

The screen displays a 3×3 grid of icons, including everyday objects such as a bottle, key, mobile phone, laptop, wallet, clock, bulb, headphones, and books. These familiar visuals reduce cognitive strain and allow users to associate items easily. Presenting the icons in a grid format supports the chunking principle, where related elements are grouped together to improve memory retention.

A countdown timer (00:05) is provided to limit observation time, encouraging focused attention and preventing over-analysis. This time constraint ensures that users rely on immediate visual memory, which is essential for testing recall ability. The clear labeling of “CHUNKING PHASE” reinforces the purpose of the screen and guides the user’s mental state toward observation rather than interaction.



Frame 3: Recall Phase – Selection Screen

Tests attention, focus, and short-term memory accuracy.

Key UI Elements

Title & Instruction

“ATTENTION AND FOCUS TEST”

“SELECT THE ITEMS YOU REMEMBER” – clearly guides user action.

Grid of Icon Choices

Multiple icons displayed in a structured grid.

Includes both previously shown items and distractor items.

Each icon has a selectable indicator (radio button/dot).

Submit Button

Allows users to confirm their selections and proceed.

Visual Design

Minimal, distraction-free layout to keep focus on recall.

How It Works

Users observe all displayed icons.

They identify and select the items they remember.

Distractors test false memory recognition.

Clicking SUBMIT completes the recall phase.

Cognitive & UX Benefits

Measures memory accuracy effectively.

Encourages focused thinking and decision-making.

Simple interaction reduces cognitive overload.

Gamified visuals keep users engaged.

ATTENTION AND FOCUS TEST

SELECT THE ITEMS YOU REMEMBER:



SUBMIT

Frame 4: Result Phase – Score & Feedback Screen

Overview

Frame 4 displays the results and feedback of the memory recall task. It helps users understand their performance and decide their next action.

Purpose of the Screen

Provides instant feedback on recall performance.

Displays the user's recall score clearly.

Offers options to continue, retry, or exit the test.

Key UI Elements

Title & Score Display

“ATTENTION AND FOCUS TEST”

“YOUR RECALL SCORE” shown prominently.

Score displayed as 4/5, indicating correct selections.

Feedback Visual

Emoji/character gives positive reinforcement.

Action Buttons

Continue – move to the next stage.

Restart – retake the test to improve score.

Exit – end the task.

How This Phase Works

System evaluates user selections from Frame 3.

Calculates the recall score.

Displays performance feedback.

User chooses the next action based on results.

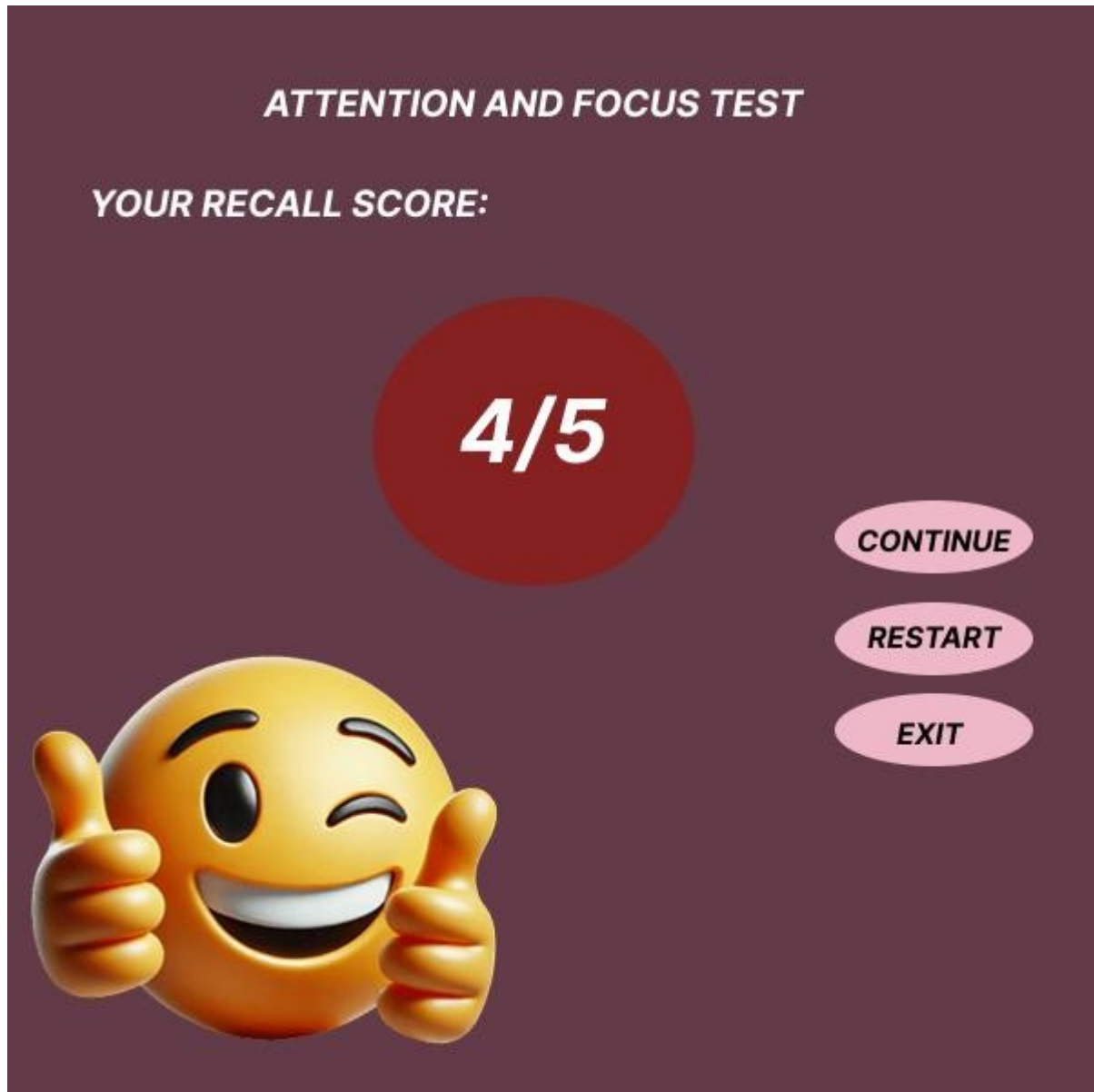
Cognitive & UX Benefits

Immediate feedback improves learning awareness.

Encourages self-assessment and improvement.

Clear navigation gives users control.

Positive visuals reduce stress and motivate users.



EXERCISE:3

NAME: Venkateshwaran . M

ROLL NO:240701584

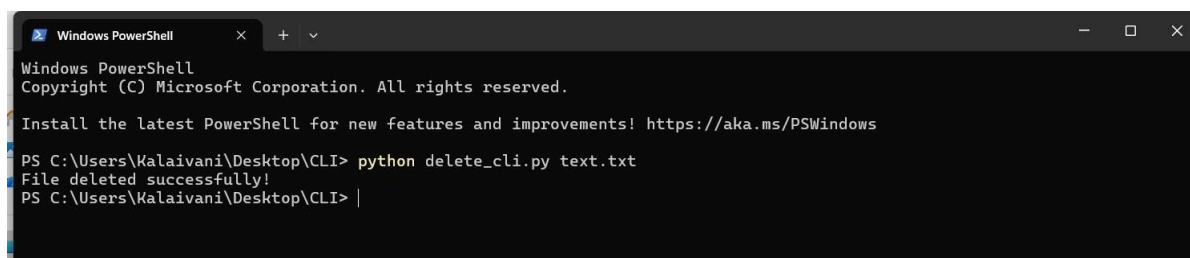
1.Command Line Interface(CLI):

Program:

```
import os, sys

if len(sys.argv) != 2:
    print("Usage: python delete.py <filename>")
else:
    try:
        os.remove(sys.argv[1])
    print("File deleted successfully")
except:
    print("File not found")
```

OUTPUT:

A screenshot of a Windows PowerShell terminal window. The title bar reads "Windows PowerShell". The terminal content shows the copyright notice for Microsoft Corporation, a message to install the latest PowerShell, and a command prompt session. The user enters the command "python delete_cli.py text.txt", and the output is "File deleted successfully!". The prompt then returns to "PS C:\Users\Kalaivani\Desktop\CLI>".

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Kalaivani\Desktop\CLI> python delete_cli.py text.txt
File deleted successfully!
PS C:\Users\Kalaivani\Desktop\CLI> |
```

2.Graphical User Interface:

Program:

```
import tkinter as tk from tkinter
```

```
import messagebox import os
```

```
def delete_file(): name = entry.get() if
```

```
os.path.exists(name): os.remove(name)
```

```
messagebox.showinfo("Success","File deleted")
```

```
else:
```

```
messagebox.showerror("Error","File not found") root = tk.Tk()
```

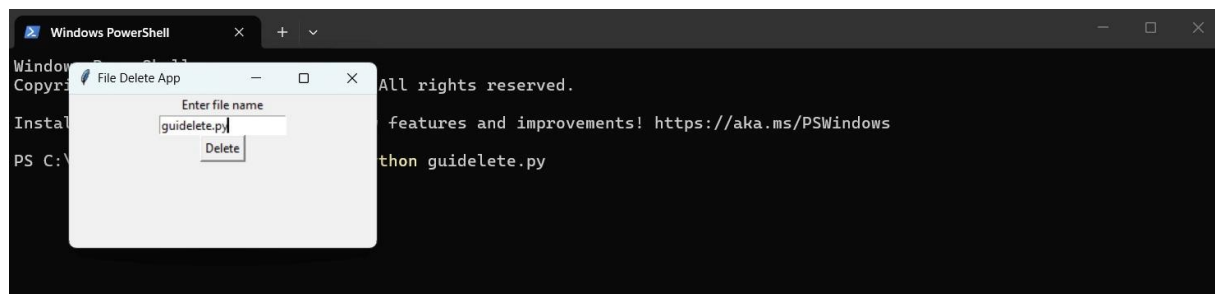
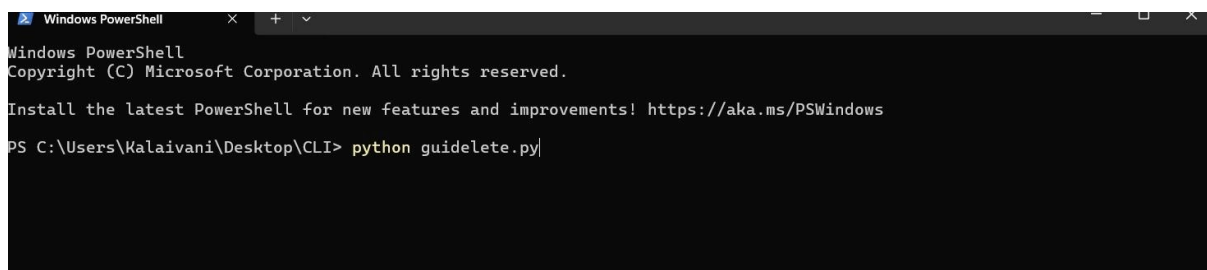
```
root.title("File Delete System")
```

```
tk.Label(root,text="Enter Filename").pack() entry =
```

```
tk.Entry(root) entry.pack() tk.Button(root,text="Delete
```

```
File",command=delete_file).pack() root.mainloop()
```

OUTPUT:



3. Voice User Interface:

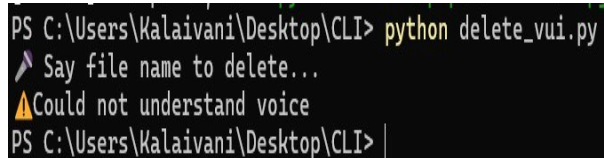
Program:

```
import speech_recognition as sr
import os

r = sr.Recognizer() with
sr.Microphone() as source:
audio = r.listen(source)

try:
    command = r.recognize_google(audio)
filename = command.split()[-1]
os.remove(filename)
    print("File deleted") except:
    print("Error in voice command")
```

OUTPUT:



```
PS C:\Users\Kalaivani\Desktop\CLI> python delete_vui.py
Say file name to delete...
Could not understand voice
PS C:\Users\Kalaivani\Desktop\CLI> |
```

4. USER SATISFACTION

COMPARISON:

Program:

def survey():

```

    print("Rate your
satisfaction for File
Delete Methods (1-5):")

    try:

        # user inputs

        cli = int(input("CLI
Delete (1-5): "))

        gui = int(input("GUI
Delete (1-5): "))

        vui =
int(input("Voice Delete
- VUI (1-5): "))

        # validate

        if not (1 <= cli <= 5
and 1 <= gui <= 5 and 1
<= vui <= 5):

            print("⚠ Please
enter ratings between 1
and 5 only.")

            return

        # display ratings

        print("\nYour
satisfaction ratings:")

        print("CLI :", cli)

```

```

    print("GUI :", gui)

    print("VUI :", vui)

    # average

    avg = (cli + gui +
vui) / 3

    print(f"\nAverage
Satisfaction Score:
{avg:.2f}")

    # best method

    ratings = {"CLI": cli,
"GUI": gui, "VUI": vui}

    best = max(ratings,
key=ratings.get)

    print("Most
Preferred Method:",
best)

    except ValueError:

        print("Invalid input!
Please enter numbers
only.")

# run function
survey()

```

OUTPUT:


```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Kalaivani\Desktop\CLI> python survey_delete.py
Rate your satisfaction for File Delete Methods (1-5):
CLI Delete (1-5): 3
GUI Delete (1-5): 4
Voice Delete - VUI (1-5): 5

Your satisfaction ratings:
CLI : 3
GUI : 4
VUI : 5

Average Satisfaction Score: 4.00
Most Preferred Method: VUI
PS C:\Users\Kalaivani\Desktop\CLI> |
```

Screen 1 – Welcome Screen

Displays the app name MindMaze with a brain icon (represents thinking & intelligence). Simple and clean design to attract users.

Contains a “Start” button to begin using the app.

Swipe instruction (“Swipe right to continue”) improves user navigation.

Acts as the entry point of the application.

Screen 2 – Dashboard Screen

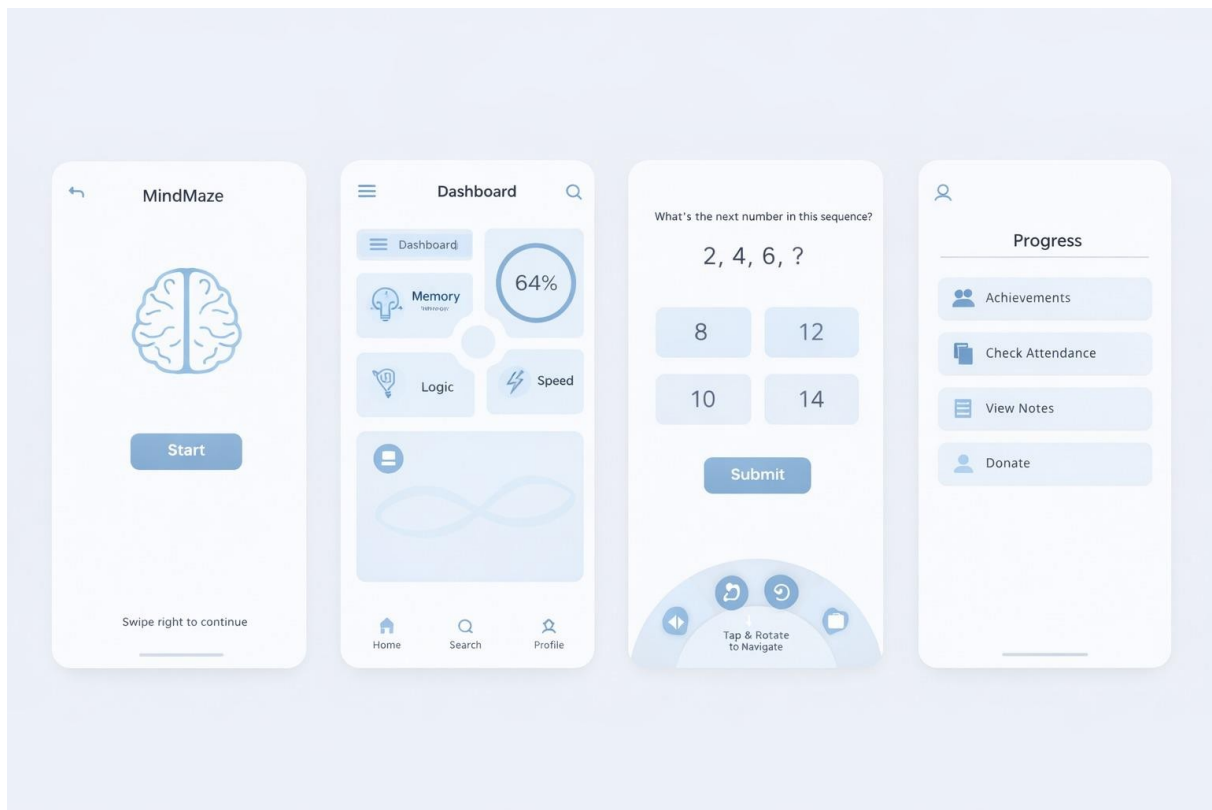
Central hub showing different features like Memory, Logic, Speed.

Displays user progress (e.g., 64% completion) using a circular progress indicator.

Organized layout with icons for easy understanding.

Bottom navigation bar (Home, Search, Profile) for quick access.

Helps users choose different brain-training activities.



Screen 3 – Quiz/Activity Screen

Displays a question (e.g., number sequence problem).

Multiple choice options (8, 12, 10, 14) for user interaction.

Includes a Submit button to check the answer.

Interactive UI with navigation controls (rotate/tap).

Designed to improve logical thinking and problem-solving skills.

Screen 4 – Progress/Profile Screen

Shows user-related options like Achievements, Attendance, Notes, Donate.

Tracks user performance and activity.

Simple list-based layout for easy navigation.

Helps users monitor their learning progress.

Includes profile icon for personalization.

Documents x Blank diagram: Lucidchart x Mobile wireframe: Lucidchart x +

lucid.app/lucidchart/44da12b5-6e26-4711-b611-4a1db1cf42d5/edit?page=0.0&invitationId=inv_b5b515f3-0c0a-4eeb-8732-6facad4f66f5#

Blank diagram Draft

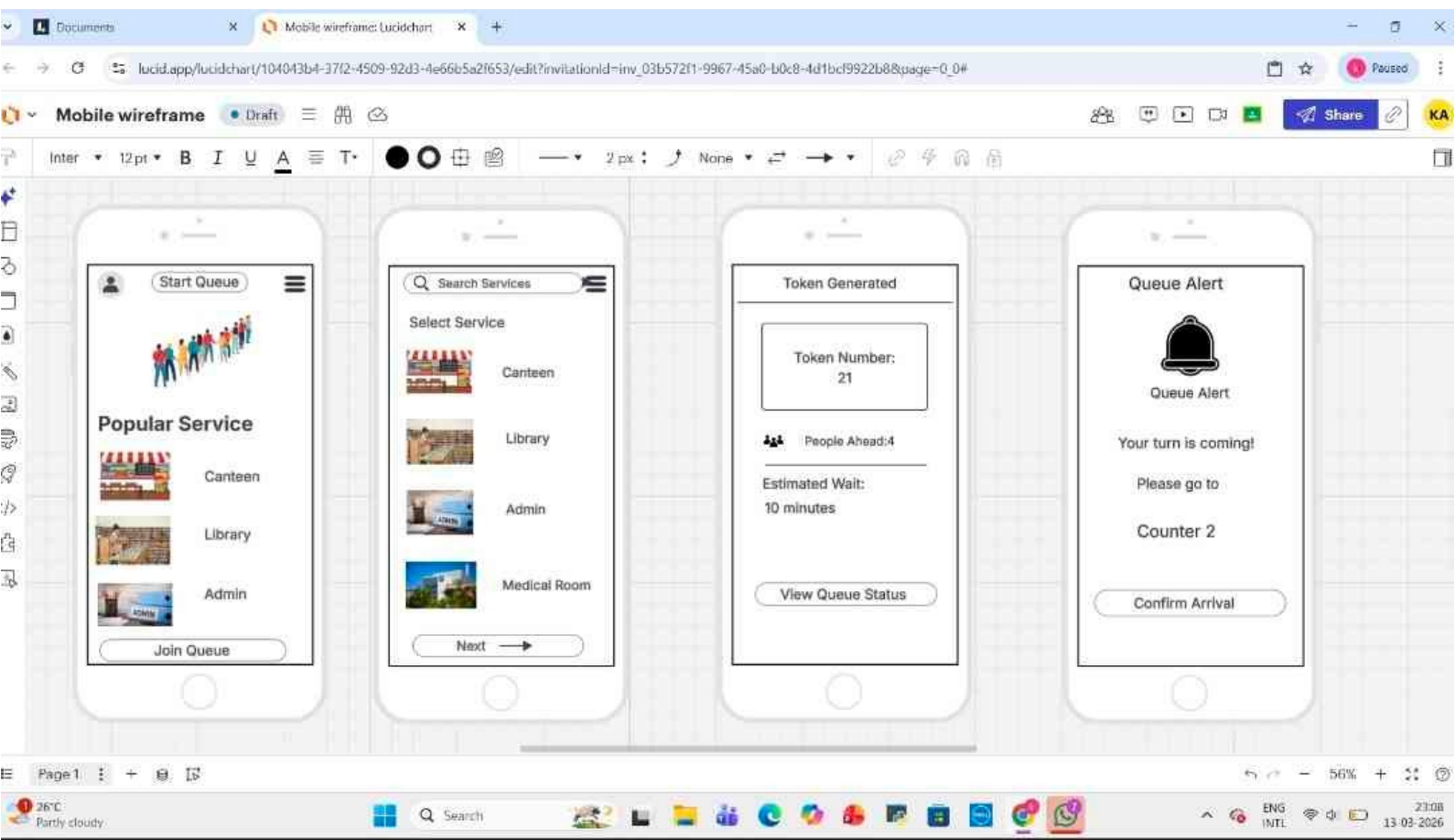
KOMIKA HAND 10 pt B I U A None 2 px

Shapes

```
graph TD; A(( )) --> B[ ]; B --> C[ ]; C --> D{ }; D --> E[ ]; E --> F[ ]; F --> G[ ]; G --> H{ }; H --> I[ ]; I --> J[ ]; J --> K[ ]; K --> L[ ]; L --> M[ ]; M --> N[ ]; N --> O(( ))
```

Page 1 7%

26°C Partly cloudy Search ENG INTL 23:09 13-03-2025



EXPERIMENT 6

NAME: Venkateshwaran . M

ROLLNO:240701584

6. Simulate the development lifecycle of a To-Do List web application using the Agile/Scrum model and implement a functional interactive interface.

1. Sprint Planning

In this stage, the project is divided into short development cycles called sprints. The team identifies the features to be built in the current sprint, such as adding tasks, marking them complete, and deleting them. User stories and acceptance criteria are defined.

2. Design and Prototyping

In this stage, wireframes and low-fidelity prototypes of the To-Do List interface are created. The layout includes an input box, an Add button, and a task list. The design focuses on clean visual hierarchy and intuitive interaction flow.

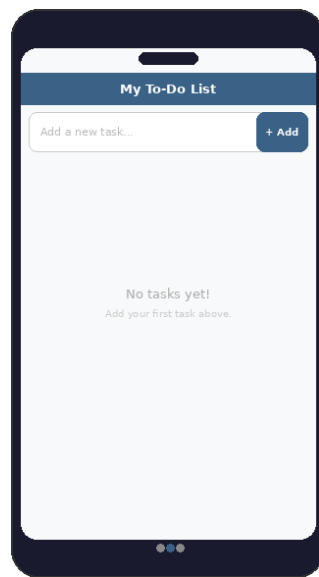
3. Development

In this stage, the actual HTML, CSS, and JavaScript code is written. Features are implemented iteratively: first the task input and display, then the mark-as-complete functionality, and finally the delete option. Each feature is tested before moving to the next.

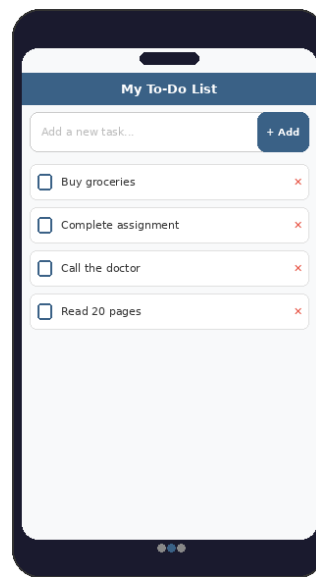
4. Sprint Review and Retrospective

In this stage, the completed To-Do List application is reviewed with stakeholders. Feedback is collected and improvements are noted for the next sprint. The team reflects on what went well and what needs adjustment to improve the development process.

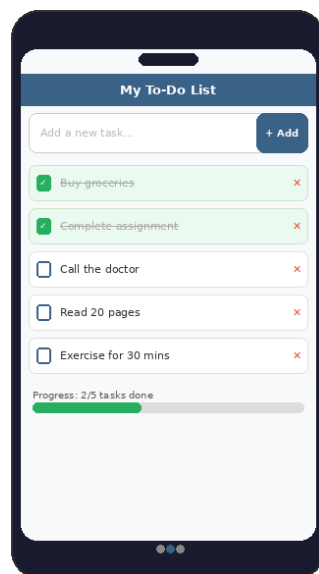
Output Screenshots:



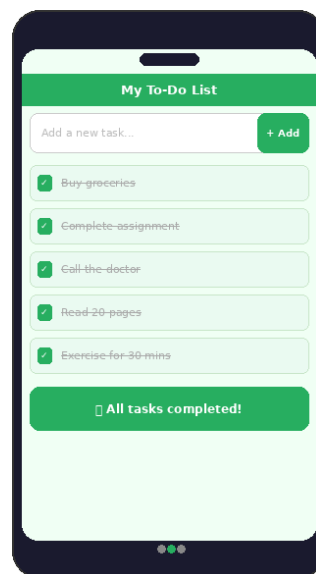
Empty To-Do List



Tasks added



Tasks marked as complete



All tasks done – Sprint Review output

EXPERIMENT 7

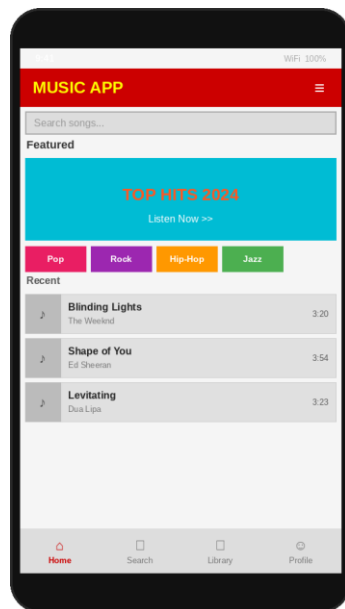
NAME: Venkateshwaran . M

ROLLNO:240701584

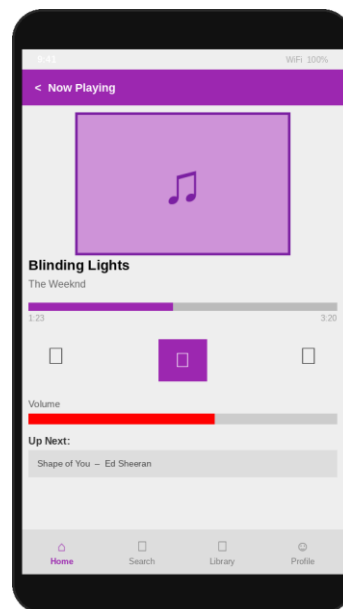
7. Design a Music Streaming application layout using different colors, text styles, and visual elements. Share the design with users through a Google Form to gather their responses. Based on the feedback received, refine and improve the design accordingly..

BEFORE REVIEW:

SCREEN 1 and 2:

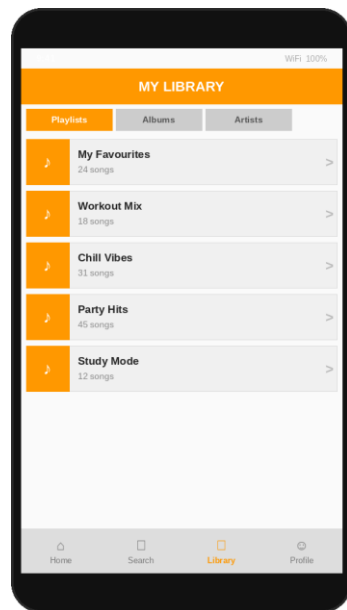


Home Screen

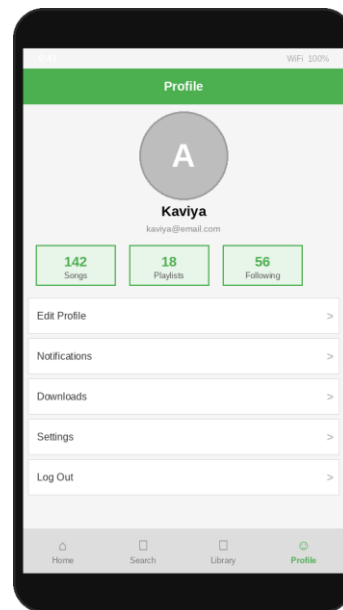


Now Playing

SCREEN 3 and 4:



Library Screen



Profile Screen

GOOGLE FORM REVIEW:

docs.google.com/forms/d/e/music-app-feedback/viewform

Music Streaming App – UI Feedback Form

Hi! Please share your honest feedback about our app design. This helps us improve your experience.

- How would you rate the overall visual design of the app?**
 - ☐ Excellent – clean and modern
 - ☐ Good – mostly appealing
 - ☐ Average – needs improvement
 - ☐ Poor – hard to use
- Is the color scheme pleasant and consistent?**
 - ☐ Yes, very consistent
 - ☐ Somewhat consistent
 - ☐ Not consistent at all
- Is the navigation bar easy to understand?**
 - ☐ Very easy
 - ☐ Somewhat easy
 - ☐ Confusing
- What feature would you like improved most? (write below)**

AFTER FEEDBACK:

Suggestion 1 — Consistent Color Theme

We replaced the mismatched multi-color headers (red, orange, purple, green) with a single unified dark purple theme across all screens. A consistent accent color (#b388ff) is now applied to all buttons, icons, active tabs, and highlights, creating a professional and cohesive visual identity throughout the app.

Suggestion 2 — Improved Typography and Text Clarity

We updated the text styles for better readability and hierarchy. Screen titles are larger and bold with strong contrast on the dark background. Supporting text like artist names and song counts uses a muted lavender tone (#b39ddb) to create a clear difference between primary and secondary information, making the interface easy to scan.

Suggestion 3 — Better Layout and Card Design

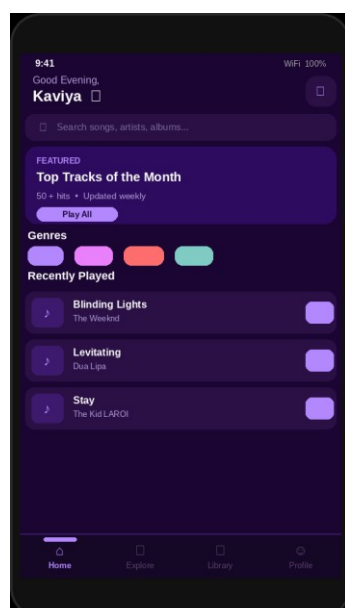
All screens were redesigned with consistent rounded card components (border-radius: 14px), proper internal padding, and equal spacing between elements. The flat, uneven rectangles seen in the before version were replaced with layered dark cards, giving the app a structured and modern look with clear visual grouping.

Suggestion 4 — Redesigned Navigation Bar

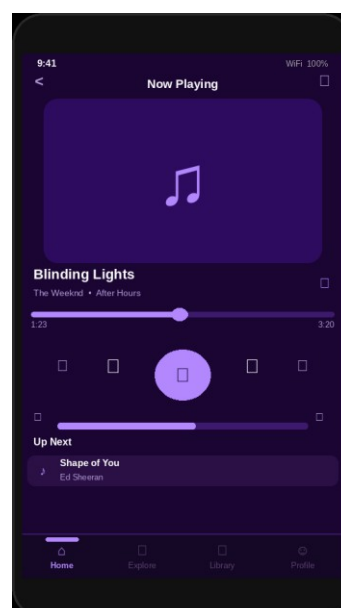
The navigation bar was redesigned with a dark background matching the app theme. The active tab now shows a colored top-line indicator and a bold accent-colored icon and label, while inactive tabs are visually dimmed. This makes it immediately clear which screen the user is on without adding clutter.

Suggestion 5 — Richer Visual Elements and Interactivity Cues

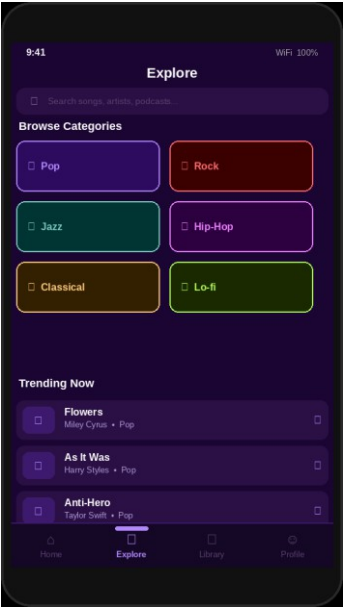
We added genre color pills on the Home screen, emoji icons for each playlist in the Library, a larger album art panel with a smooth progress bar and draggable thumb on the Now Playing screen, and a color-coded category grid on the Explore screen. These changes make the app feel engaging, expressive, and easy to navigate.



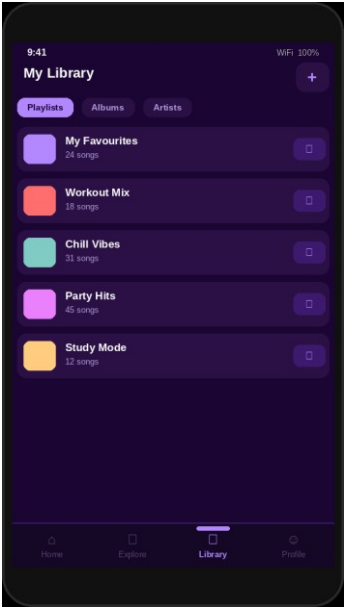
Home Screen



Now Playing



Explore Screen



Library Screen

EXPERIMENT:8

NAME: Venkateshwaran . M

ROLLNO:240701584

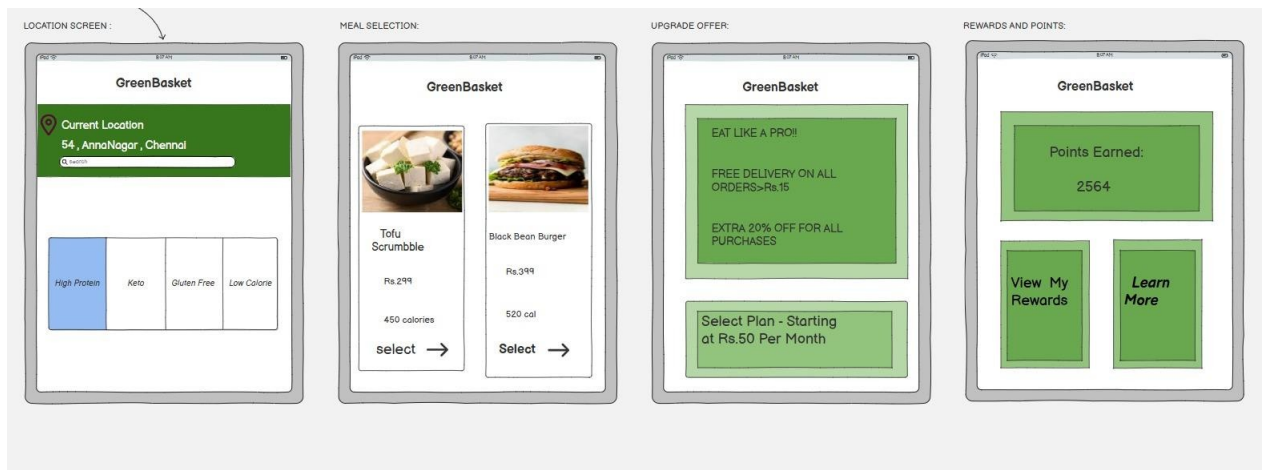
8.create storyboards to represent the user flow for your application using Balsamiq

1. Location Screen

- Displays **current delivery location** (e.g., Anna Nagar, Chennai).
 - Includes a **search bar** for changing location.
 - Shows **diet categories** like:
 - High Protein
 - Keto
 - Gluten Free
 - Low Calorie
 - Helps users quickly filter meals based on preferences.
-

2. Meal Selection Screen

- Shows food items with:
 - Image
 - Name (e.g., Tofu Scramble, Black Bean Burger)
 - Price
 - Calories



- Each item has a **“Select” button** for easy ordering.
- Clean card-based layout improves readability.

3. Upgrade Offer Screen

- Promotes **premium plan / offers**:
 - Free delivery
 - Discounts (e.g., 20% off)
- Includes a **subscription option** (starting at ₹50/month).
- Encourages users to upgrade for benefits.

4. Rewards & Points Screen

- Displays **earned reward points** clearly.
- Includes action buttons:
 - View My Rewards
 - Learn More

- Motivates users through **gamification and loyalty system**.

NAME: Venkateshwaran . M

ROLL.NO:240701584

Experiment 9: Form Validation using HTML, CSS & JavaScript

AIM:

To design input forms that validate data such as email and phone number, and display appropriate error messages to the user using HTML, CSS, and JavaScript with the **Validator.js** library.

1. Setting Up the HTML Form

The first step is to create the skeleton of the form using plain HTML. We add two `<input>` fields — one for *email* and one for *phone number* — along with a submit button and placeholder `<span>` tags where error messages will later appear. The Validator.js CDN link is included inside `<head>` so the library is available for Step 3.

```
HTML
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Form Validation</title>

  <script src="https://cdnjs.cloudflare.com/ajax/libs/validator/13.11.0/
validator.min
.js"></script>
</head>
<body>

  <h2>Contact Form</h2>

  <form id="contactForm">

    <label for="email">Email Address</label>
    <input type="text" id="email"
      placeholder="Enter your email" />
    <span id="emailError"></span>

    <label for="phone">Phone Number</label>
    <input type="text" id="phone"
      placeholder="Enter your phone number" />
    <span id="phoneError"></span>

    <button type="submit">Submit</button>

  </form>

</body>
</html>
```

OUTPUT – STEP 1

Contact Form

Email Address Phone Number Submit

Explanation: The output shows a plain, unstyled HTML form with two input boxes (Email & Phone) and a Submit button. There is no styling or validation yet — the browser uses its default font and layout. The `<span>` tags for error messages are invisible at this stage because they contain no text.

2. Styling the Form with CSS

Now we add CSS to transform the bare form into a clean, modern card-style layout. Key styling choices: a card with rounded corners and a drop shadow, uppercase labels, smooth focus transitions on inputs, and a solid-color submit button with a hover effect. We also pre-define `.valid`, `.invalid`, and `.error-msg` classes that JavaScript will apply in Step 3.

```
body {
  font-family: 'Segoe UI', Arial, sans-serif;
  background: #f0f4f8;
  display: flex;
  justify-content: center;
  padding: 50px 16px;
  min-height: 100vh;
}

form {
  background: #fff;
  border-radius: 12px;
  box-shadow: 0 6px 28px rgba(0,0,0,0.12);
  padding: 30px 28px;
  width: 100%;
  max-width: 420px;
}

label {
  display: block;
  font-size: 12px;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.5px;
  color: #444;
  margin-bottom: 6px;
}

input {
  width: 100%;
  padding: 11px 14px;
  border: 2px solid #d8d4ce;
  border-radius: 8px;
  font-size: 14px;
  outline: none;
  background: #faf9f7;
  transition: border-color 0.2s;
  margin-bottom: 4px;
}

input:focus { border-color: #2c5f8a; background: #fff; }
```

```

input.valid { border-color: #28a745; }
input.invalid { border-color: #dc3545; }

.error-msg
{ display: none;
font-size: 12px;
color: #dc3545;
margin-bottom: 12px;
}
.error-msg.show { display: block; }

button
{ width:
100%;
padding: 12px;
background: #2c5f8a;
color: #fff;
border: none;
border-radius: 8px;
font-size: 15px;
font-weight: 600;
cursor: pointer;
transition: background 0.2s;
margin-top: 8px;
}
button:hover { background: #1e4468; }

```

OUTPUT – STEP 2

Contact Form

Please fill in the details below.

Email Address

Phone Number

Submit

Explanation: After applying CSS the form appears as a clean white card centred on a light grey background. Labels are small and uppercase; input fields have rounded corners and a smooth blue focus border. The submit button is styled with the primary brand colour (#2c5f8a) and darkens on hover. Error message spans are still hidden (display:none) — they will be toggled by JavaScript.

3. Adding JavaScript Validation with Validator.js

Finally we wire up the validation logic. On every input event we call `validator.isEmail()` and `validator.isMobilePhone()` to check the entered values in real time. If a value is invalid the corresponding

 becomes visible, the input border turns red, and the submit button is blocked. When both fields pass, the borders turn green and the form is allowed to submit.

JAVASCRIPT

```
const emailInput = document.getElementById('email');
const phoneInput = document.getElementById('phone');
const emailError = document.getElementById('emailError');
const phoneError = document.getElementById('phoneError');
const form = document.getElementById('contactForm');

emailInput.addEventListener('input', () => {
  const val = emailInput.value.trim();

  if (val === '') {
    setInputState(emailInput, emailError, '', '');
  } else if (validator.isEmail(val)) {
    setInputState(emailInput, emailError, 'valid', '');
  } else {
    setInputState(emailInput, emailError,
      'invalid',
      '⚠ Please enter a valid email address.');
```

```
  });

phoneInput.addEventListener('input', () => {
  const val = phoneInput.value.trim();

  if (val === '') {
    setInputState(phoneInput, phoneError, '', '');
  } else if (validator.isMobilePhone(val, 'any', { strictMode: false })) {
    setInputState(phoneInput, phoneError, 'valid', '');
  } else {
    setInputState(phoneInput, phoneError,
      'invalid',
      '⚠ Enter a valid phone number (e.g. +91 9876543210).');
```

```
  });

function setInputState(input, errorEl, state, message) {
  input.classList.remove('valid', 'invalid');
  if (state) input.classList.add(state);

  errorEl.textContent = message;
  errorEl.classList.toggle('show', message !== '');
}

form.addEventListener('submit', (e) => {
  e.preventDefault();

  const emailOk = validator.isEmail(emailInput.value.trim());
  const phoneOk = validator.isMobilePhone(
    phoneInput.value.trim(), 'any', { strictMode: false });

  if (!emailOk) setInputState(emailInput, emailError,
    'invalid', '⚠ Email is required and must be valid.');
```

```
  if (!phoneOk) setInputState(phoneInput, phoneError,
    'invalid', '⚠ Phone number is required and must be valid.');
```

```
if (emailOk && phoneOk)
{ document.getElementById('result').className = 'success';
  document.getElementById('result').textContent =
    '✅ Form submitted successfully!';
}
});
```

OUTPUT – STEP 3 (INVALID INPUT STATE)

Contact Form

Please fill in the details below.

Email Address

Phone Number

Submit

OUTPUT – STEP 3 (VALID INPUT + SUCCESS)

Contact Form

Try entering your details below!

Email Address

Phone Number

Submit

Explanation (Error State): When the user types an incomplete email like *john.doe@* or a short number like *123*, Validator.js immediately returns false. The JavaScript adds the invalid class (red border + red glow) to the input and makes the visible with a warning message. The form cannot be submitted.

Explanation (Valid State): Once a correct email (e.g. *user@example.com*) and a valid phone number (e.g. *+91 9876543210*) are entered, the borders turn green (valid class), error spans hide, and clicking Submit shows a green success banner. The live form above is interactive — try it!